

The RAPP/1 organism, on one page

Read it top (6) to bottom (0). Left: what comes in. Right: what goes out or across.

RAPP/1 (LTS): in force

newest: not in RAPP/1 yet

in force

specified

experimental

candidate

planned

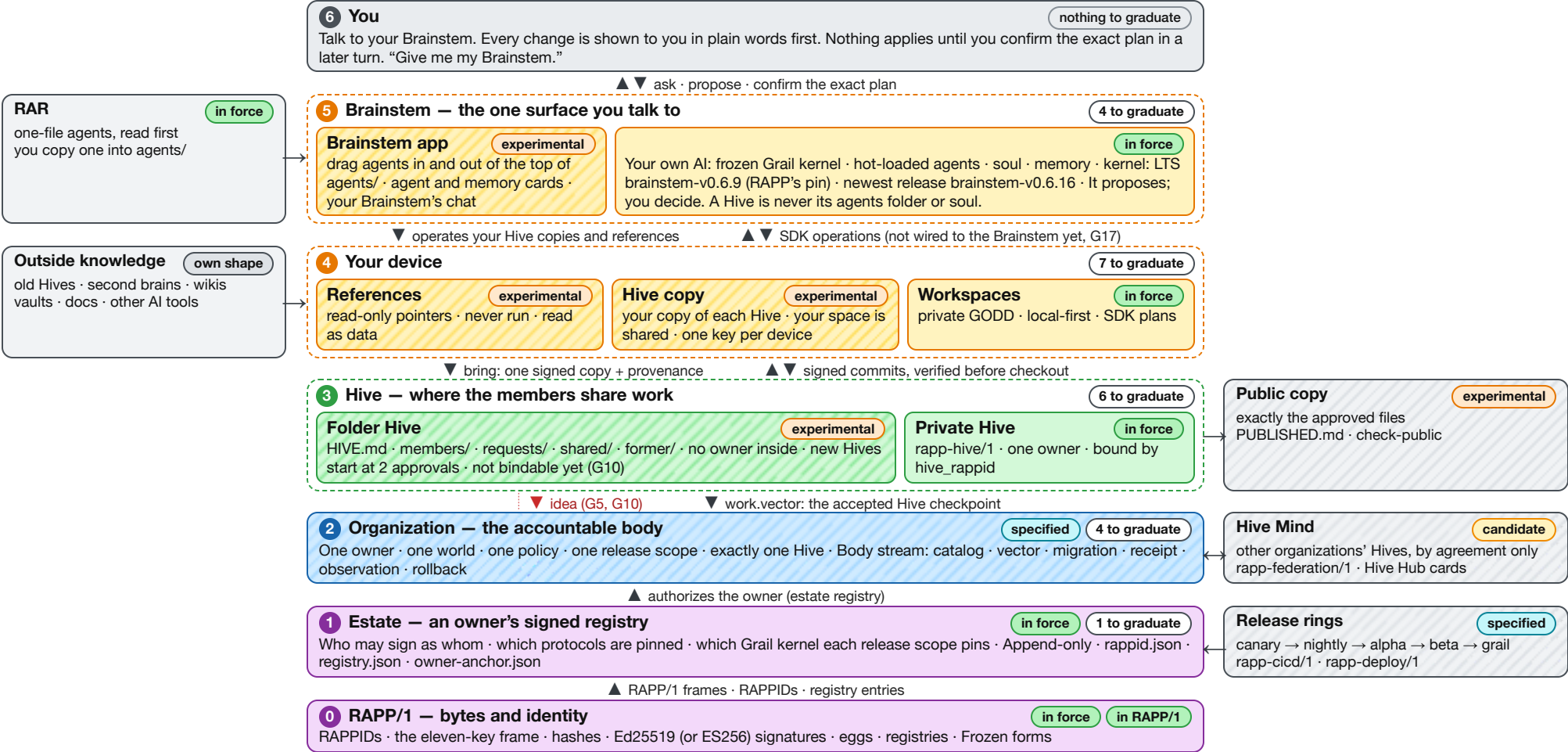
gap

idea

open

proposed

own shape



Dogfood: the RAPP Hive, our public example

experimental

HIVE.md

members: the maintainers (AIs are tools)

shared/organism/canon/

one canonical fact per file, hash-stamped

shared/organism/drift/

one verdict per repo: aligned or drifted

shared/organism/map/

layers 0-6 · crossings · this page

shared/organism/gaps/

one file per gap

(your device)

references: every estate repo, read-only

1 Canon

one fact, once, from its repo

2 Sweep

rapp_check.py, not in the Hive

3 Record

one verdict per repo, signed

4 Fix upstream

the next sweep flips it

Built locally; public copy: kody-w/rapp-hive-public.

Its first sweep ran rapp-1's own rapp_check.py over 10 public estate repos: 7 were compliant or clean and 3 showed drift (non-conformant eggs in RAPP and RAR; frozen frames in rapp-model-hive's main that cannot be signature-checked, gap G8). Each verdict is one signed file. How it scales: the same one agent and the same rules run the 6-person Contoso model and 10 real repos, on the way to about 30.

RAPP/1 LTS

page 2: the lock-in

One LTS release of the whole stack: one estate release scope that pins every component. Everything in it is in force, and nothing in it is experimental. Newest releases get their own successor scopes. Experiments live only there until they graduate.

1 Decide · you

G8 G9 G18 G19

2 Zero drift, clean pull · engineering 4 steps, no gaps

3 Close the specification gaps · spec owner

G1 G2 G3 G4 G5 G6 G7 G10 G11 G22

4 Switch it on · estate owner

G15 G16 G17

5 Graduate into RAPP/1 · engineering

G12 G13 G14 G20

What holds everywhere

- RAPP/1 wins, then rapp-work/1.
- Transport carries; signatures decide.
- In by signed copy, out by approved copy.
- Other people's text is data.
- Propose, confirm, apply.
- Accepted history is never rewritten.
- The Brainstem is the one surface you talk to.
- Only top-level agents/*_agent.py files are live.

RAPP/1 LTS: what it takes to lock the whole thing

RAPP/1 is the one LTS release people pull. Five phases lock it, in order, with the owner’s decisions first.

RAPP/1 (LTS): in force

newest: not in RAPP/1 yet

in force

specified

experimental

candidate

planned

gap

idea

open

proposed

own shape

Your decisions first · phase 1, Decide · you, about an hour

1. merge RAPP proposal 0001 (G18), a documentation fix only: it is yours because RAPP reserves constitution merges for the maintainer (Articles XXVIII.4 and XXX.2)
2. accept or refuse RAPP proposal 0002 (G19), Tier 2 loading: a draft on experimental/proposal-0002-tier2-parity
3. ratify the RAPP Work Constitution and its Part V (G8, G9)

RAPP/1 LTS is

- One LTS release of the whole stack: one estate release scope that pins every component.
- Everything in it is in force, and nothing in it is experimental.
- Newest releases get their own successor scopes. Experiments live only there until they graduate.

It pins: `the LTS kernel brainstem-v0.6.9` `rapp-hive/1` `rapp-work/1` `RAPP Workspace/1`
`rapp-work-sdk/1` `rapp-registry/1.0` `rapp-cicd/1` `rapp-deploy/1`
the agents and apps that ship with it

RAPP/1 §11.1: a release scope's pin is “a permanent compatibility anchor, not a moving release channel”, and “A successor uses a new `release_scope`”. §13.3: no two `grail-kernel` entries share a scope.

2 · Zero drift, clean pull

engineering

- fix RAPP’s and RAR’s non-conformant eggs, and RAR’s bounded scan
- give rapp-model-hive’s frames a trusted anchor
- re-sweep until all 10 repos are clean (the dogfood)
- keep the default pull clean: default branches and the LTS install carry only in-force parts; experiments stay on experimental branches or in newest

3 · Close the specification gaps

spec owner

- rapp-hive/1 (G1, G6)
- rapp-work-sdk/1 (G2, G3, G4, G7)
- rapp-work/1 (G5, G10, G11, G22)
- a gap that cannot close in time waits for a later RAPP, with the owner’s sign-off

G1 G2 G3 G4 G5 G6 G7 G10 G11
G22

Each layer, top to bottom: in RAPP/1, or what it needs to graduate

6 You	nothing to graduate								
5 Brainstem	4 to graduate	G17	G18	G19	G20				
4 Your device	7 to graduate	G2	G3	G4	G7	G13	G14	G22	
3 Hive	6 to graduate	G1	G6	G8	G9	G12	Public copy		
2 Organization	4 to graduate	G5	G10	G11	G16				
1 Estate	1 to graduate	G15							
0 RAPP/1	in RAPP/1								

4 · Switch it on

estate owner

- sign the RAPP/1 LTS release scope: its LTS `grail-kernel` entry, plus a successor scope for the newest kernel (G15)
- sign the protocol pins
- activate rapp-work/1 (G16): the Organization must be in force for RAPP/1 to be healthy throughout
- give the Brainstem an SDK agent (G17)

G15 G16 G17

5 · Graduate into RAPP/1

engineering, through the rings

- graduate each newest part through the rings: Folder Hive, Hive copy and the Hive agent (G14), References (G13), Hub cards (G12), Public copy, the Brainstem app (G20)
- then Hive Mind, once an estate accepts `rapp-federation/1`, and the Release rings, once phase 4 activates them and a graduation uses them
- merge what graduated; the rest stays out of what people pull
- tag the lock once the clean-pull check passes again; sweep for drift on a schedule

G12 G13 G14 G20

2 · Keep the default pull clean: files on each default branch that mention “experimental” (git grep -l -i experimental origin/HEAD), 2026-09-25: RAR 50 RAPP 45 rapp-workspace 26 rapp-hive-hub 19

rapp-installer 9 rapp-model-hive 8 hive-hub 6 lispy 1 rapp-1 0 rapp-work 0 Raw word hits, and some are legitimate, such as folder names: mentions, not problems. Run it again at lock time.

Gap register: each gap, its phase and who acts

G1	A rapp-hive/1 roster can never change	open	3 · spec owner	G12	Hive Hub dial records did not describe folder Hives	experimental	5 · engineering
G2	SDK plans cannot move a file	open	3 · spec owner	G13	Outside knowledge had no way in	experimental	5 · engineering
G3	SDK discovery misses <code>*_agent.py</code>	open	3 · spec owner	G14	The Hive agent’s path to the production ring	open	5 · engineering
G4	SDK migration refuses repository-seeded Hives and long world ids	open	3 · spec owner	G15	No estate declares the Brainstem’s kernel	open	4 · estate owner
G5	One owner only, so co-equal groups do not fit	idea	3 · spec owner	G16	No estate has activated canonical rapp-work/1	open	4 · estate owner
G6	Implementations refuse owner succession	open	3 · spec owner	G17	The Brainstem does not call the RAPP Work SDK	idea	4 · estate owner
G7	SDK verification accepts an edited instruction file	open	3 · spec owner	G18	RAPP’s Constitution describes a two-day experiment	proposed	1 · you
G8	Hive members are names bound to keys, not RAPPIDs	proposed	1 · you	G19	The cloud Brainstem loads agents differently	proposed	1 · you
G9	A git commit is not authority in canonical rapp-work/1	proposed	1 · you	G20	The Brainstem app’s path to the production ring	open	5 · engineering
G10	An organization cannot bind a folder Hive	idea	3 · spec owner	G22	“Workspace” names two things	idea	3 · spec owner
G11	“Organization” means two things	idea	3 · spec owner				